University of Economics, Prague

Faculty of Informatics and Statistics

IMPLEMENTATION OF ARTIFICIAL INTELLIGENCE SERVICES IN CLOUD CALL CENTER AGENT PANEL

[BACHELOR]

Study programme: Apllic

Field of study: Aplikovaná informatika

Author: Adam Kovářík

Supervisor: whole name of your supervisor Ing. Filip Vencovský, Ph.D.

Prague, December 2021

Acknowledgement

I hereby wish to express my appreciation and gratitude to the supervisor of my thesis, Ing. Filip Vencovský, Ph.D.

Abstract

Most call centers currently require expensive hardware and office spaces to operate. Also, agents who answer customers' questions must manually search for answers in files commonly scattered across multiple locations. This prolongs answering time which, resulting in longer calls and thus larger waiting queues. With new cloud technologies, it is possible to create a call centre hosted purely in cloud environment that can be used in production without the need for any specialized hardware. Therefore agents can work remotely since they can answer calls via web browsers or even via Microsoft Teams.

Our department has already created a Proof of Concept web application whose main purpose is to see if agents may receive real-time article recommendations, but it also supports call answering, real-time call transcription, and translation. This solution is working, but to provide article suggestions, it must use Amazon Kendra, which is too expensive to be used only for this purpose.

Since then, Amazon has released a new solution called Amazon Connect Wisdom which is interconnected with Amazon Connect, and it can recommend FaQ articles during calls based on caller's questions. The goal of this thesis is to create Proof of Concept web application that has the same features as the application described earlier, but it must use Amazon Connect Wisdom instead of Amazon Kendra. It should also compare Amazon to other cloud providers and evaluate if Amazon is the right provider.

The thesis has two parts. In the first part, NLP and its history are introduced. Multiple NLP cloud providers are evaluated, and an ideal provider is recommended.

The second part focuses on building a web application for agents. Firstly, the architecture of the overall solution is described. After that follow chapters regarding infrastructure set up, cloud services configuration. The next chapter describes back-end and front-end code. Then review from cloud expert Matthias Schardt is noted and the thesis ends up with a conclusion if this PoC has been successful and recommendations if the project shall continue above PoC.

amet, consectetuer, dolor, Lorem ipsum, sit.

JEL Classification

amet, consectetuer, dolor, Lorem ipsum, sit.

Content

Introduction

Most call centers currently require expensive hardware and office spaces to be able operate. Also, agents who answer customers questions must manually search for answers in files that are commonly scattered across multiple locations. This prolongs answering time which results in longer calls and thus larger waiting queues. With new cloud technologies it is possible to create call center hosted purely in cloud environment that can be used in production without need of any specialized hardware and also agents can work remotely since they can answer calls via web browser or even via Microsoft Teams. Our department has already created Proof of Concept web application which main purpose is to see if it is possible that agents receive real time article recommendations, but it also supports call answering, real time call transcription and translation. This solution is working but to provide article suggestions it must use Amazon Kendra which is too expensive to be used only for this purpose. Since then, Amazon has released new solution called Amazon Connect Wisdom which recommends articles in real time.

Goal of this is thesis is to create Proof of Concept web application that has same features as application described earlier but it must use Amazon Connect Wisdom instead of Amazon Kendra. It should compare Amazon to other cloud providers and evaluate if Amazon is right provider.

1 Natural language processing

This chapter introduces the natural language processing field and briefly describes its older history and more recent deep learning era.

1.1 Introduction

Natural language processing (NLP) is a branch of computer science, artificial intelligence, and linguistic that aims to provide computers the ability to understand and generate text or speech in the same way humans do.

"Natural Language Processing is a theoretically motivated range of computational techniques for analysing and representing naturally occurring texts at one or more levels of linguistic analysis for the purpose to achieve human-like language processing for a range of tasks or applications." (Liddy, 2001)

NLP research and development can be categorized into the following five areas (Joseph et al., 2016):

- Natural Language Understanding
- Natural Language Generation
- Speech or Voice recognition
- Machine Translation
- Spelling Correction and Grammar Checking

Real-world application of NLP can be found in machine translation, natural language text processing, and summarization, user interfaces, multilingual and cross-language information retrieval (CLIR), speech recognition, text to speech, speech to text, expert systems (

1.2 Brief history

Origins of natural language processing can be dated back to late the 1940s with the work of Weaver and Booth in the machine translation field and Weaver's influential

memorandum on translation of 1949 brought attention of scientific community (Joseph et al., 2016). Besides these early birds research on NLP began in the 1950s where automatic translation from Russian to English was performed in a limited experiment and exhibited in the IBM-Georgetown Demonstration of 1954, also the journal MT (Mechanical Translation) began publication in 1954 (Jones, 1994).

Results from early experiments of machine translation have been poor, and research realized that theory of language is needed. In 1957 Chomsky published Syntactic Structures which gave more insights on how linguistics could help with machine translation (Liddy, n.d.). Major research done in this era focused on syntax and machine translation was done mainly as lookup in dictionary which did not capture long-distances dependencies that appear in languages such as German (Jones, 1994).

The 1950s and early 1960s were full of optimism and enthusiasm. However, access to machines was often restricted, code could be written only in assembly, and computing power was extremely low. Compared to the expectations, progress was being made very slowly, which led to the 1966 ALPAC Report, which almost killed the research of MT because it concluded that MT wasn't even close to the achievement and led to funding cuts, especially in the USA (Jones, 1994).

After the ALPAC cloud, some significant work has been done in theory issues and the development of prototype systems. The focus in theoretical research in the late 1960s and early 1970s was on meaning representation (Liddy, 2001). Examples of theoretical work done Chomsky's 1965 transformational model of linguistic, case grammar of Fillmore, semantic networks of Quillian, and conceptual dependency theory of Schank, which explained syntactic anomalies and provided semantic representations; Formalism's representation which included Wilks' preference semantic and Kay's functional grammar.

Building upon growing theory, many systems were built in this era of NLP. Wood's LUNAR (Woods, 1978) was a question answering system, which provided answers to questions about the chemical analyses of the Apollo 11 moon rocks. Winograd's SHRDLU had a simple conversation with user about a small world of objects. It could answer questions about the objects and execute the user's commands, such as moving

objects around (Standford, n.d.). Weizenbaum's ELIZA (Weizenbaum, 1966) was a program that could have a conversation with a user using pattern matchmaking.

In the late 1970s and 1980s, it became clear that it was much harder to build an extensive NLP program than anticipated, even for small domain topics. NLP research in this era was influenced by the development of grammatical theory and AI reasoning, which resulted in major progress regarding syntax. (Jones, 1994). Machine translation was revived mainly in Japan, where several translation products appeared (Jones, 1994).

In the late 1980s and 1990s, the rapidly increasing supply of text materials made research easier. NLP shifted focus into research and development in message processing and text processing which extracts key concepts from a full text (Jones, 1994). It was the era of statistical machine learning.

1.3 Deep learning and NLP

"The goal of statistical language modeling is to predict the next word in textual data given context; thus we are dealing with sequential data prediction problem when constructing language models." (Mikolov et al., 2010)

In 2003 first neural language model was created (Bengio et al., 2003). It was made of a one-hidden layer feed-forward neural network with fixed-length context and has achieved state-of-the-art results in longer sentences (Bengio et al., 2003). Word embedding was used for the first time to fight the curse of dimensionality (Bengio et al., 2003). Word embedding is matrix in which each vocabulary word is associated with a word feature vector. Words that have similar vectors then should also have a similar meaning. Feature vectors are trained but can also be initialized. (Bengio et al., 2003)

The issue with Bengio's model is that it has to use fixed-length context, which means that neural networks (RNNs) see only a few preceding words, but humans, on the other hand, use wide longer context, and it helps them to understand the sentences (Mikolov et al., 2010). RNNs solve this issue by using a recurrent connection that does not limit the content's size. The problem of recurrent neural networks is that they are hard to train using backpropagation because of the vanishing gradient problem. Better optimization

algorithms can address this problem but then training of these models has significantly increased computational costs, which makes RNNs less attractive for large training data (Sundermeyer et al., 2012).

Long Short-Term Memory (LTSM) is modifying network architecture to avoid vanishing gradient problems while the training algorithm remains the same (Sundermeyer et al., 2012)

2 Evaluation of cloud providers

In this chapter I compare Amazon against its competitors Google, and Microsoft. I have researched if they offer call center solutions and I have also compared relevant NLP solutions from each provider.

2.1 Google

Google has a solution called Contact Center AI. It leverages Dailogflow to enhance call center capabilities of speech-to-text, text-to speech and intent recognition. The core of Contact center AI is new version of DialogFlow called DialogFlow CX. It can also provide hints to agent's based on documents in knowledgebase. Solutions based on Google's contact center can be used either by calling via telephone or by chat. ("Contact Center AI," n.d.)

2.1.1 DialogFow CX

"A Dialogflow CX agent is a virtual agent that handles conversations with your end-users. It is a natural language understanding module that understands the nuances of human language."("Dialogflow CX basics," n.d.)

Dialog Flow CX is a low code platform in which conversation is defined. The main building blocks are flows and pages. The goal of flows is to gather relevant information from the user and then either answer his question directly, perform an action in the backend system or connect to a human. Multiple flows can be created based on a number of different conversational topics that the Dialog Flow CX agent should handle. Flows are defined by pages. Talking to the bot is done in turns, meaning one turn is when the user speaks, and another turn is when the bot speaks. What the should bot do in his turn is defined on the page.

The key concept of AI agents is to translate human spoken language into computer-readable data. This is done via intents and entities. The intent is to capture user intention and is driving the conversation flow. Intents are created by adding multiple sentences that express the same intention into Dialog Flow CX. Pretrained NLP model that trains on these examples and can then recognize intentions of sentences with different wording that was

used in training examples. Entities are used for keywords extraction from the sentences such as dates, times, colors, email addresses.

Automation is driving business savings, and it can be in Dialog Flow CX using webhooks. Data extracted from conversation can be passed to webhook to validate the data, retrieve data from backend systems and transform them using business logic or trigger an action in the backend system.

2.2 Microsoft

Microsoft Teams can be used by organizations as a contact center, but Microsoft itself doesn't provide contact center solution. (cazawideh, n.d.) Comprehensive AI based solution has been built by companies such as Anywhere365 on top of Microsoft Teams.

Anywhere365's solution is an omnichannel contact center that users can reach it via phone but also via multiple channels such as webchat, email, WhatsApp, and other channels ("Cloud Contact Center Solution | CCaaS," n.d.).

Dialogs are build using Dialog studio which is drag and drop dialog builder. Conversation can be enriched using Microsoft Luis and Microsoft QnA Maker which are integrated. ("Omnichannel Dialogue Cloud Platform," n.d.)

2.2.1 Microsoft Luis

Microsoft Luis is a machine learning-based service that uses NLP to extract intents and entities from text. It does not have a conversation dialog build as Google Dialog Flow CX or Amazon has, but it has some additional capabilities in NLP. Entities can be extracted not only based on keyword matching but Luis can be trained to look for words in a specific part of sentences. Also, it uses active learning to improve custom models continuously.

2.2.2 Microsoft QnA Maker

As the name suggests, Microsoft QnA Maker is used to answer simple questions. It can be trained by uploading an excel or CSV with question pairs prefilled. It can also extract QnA pairs from web pages and pdf manuals. QnA Maker uses active learning as well, which works so that a list of the possible answer is provided to the user, and if he

selects one of, his original question is saved to the database. Users managing QnA Maker can later go through saved questions and decide which ones should be used for the retraining model and which ones not.(nerajput1607, n.d.)

2.3 Amazon

Amazon offers Amazon Connect, which is an omnichannel cloud contact center ("What is Amazon Connect? - Amazon Connect," n.d.)This means that chat and voice contact can be used by customers dynamically. Amazon Connect has a strong support of NLP. It supports speech-to-text and text-to-speech out of the box, but it is possible to also add an Amazon Lex bot for greater support, such as intent identification.

Conversation flow is created in Contact Flow designer, which is a low code drag and drops editor. Conversation flow is made of contact blocks that define logic for each step in conversation. ("Contact flows - Amazon Connect," n.d.). Contact block can also invoke AWS Lamba to work with data from backend systems.

2.3.1 Amazon Lex

Similar as Google Assistant Amazon Lex uses intents to detect user intention. But instead of entities it uses slots to extract additional data from sentences.

2.3.2 Amazon Wisdom

Another NLP feature is Amazon Wisdom. Agents can save the time of answering customer questions. Instead of navigating between different sources of information, agents can ask wisdom questions in the same way as customers ask them. Wisdom then searches connected knowledge articles, wikis, and FaQs and provides the best answers back to the agent. This can also be done in real-time without agents typing the question by using real-time speech analytics to detect customer issues during calls and provide agents recommendations and answers ("Amazon Connect Wisdom – Amazon Web Services," n.d.)

2.3.3 AWS Lamba

Amazon connect also well suited for process automation. Amazon connect supports dynamically invoking AWS Lamba functions. AWS Lambda is a serverless compute service that can run code without provisioning or managing servers ("Serverless

Computing - AWS Lambda - Amazon Web Services," n.d.). It enables connections to the back-end system for data retrieval, transformation, and action performance. Retrieved data can be provided back to customers via speech-to-text.

2.4 NLU Comparison

In this subchapter I have picked relevant NLU services from each provider and evaluated by 4 categories which are general NLU capabilities, support of automated testing, language support and pricing. Each category is evaluated individually and at the end overall results are compared.

2.4.1 NLU capabilities

In intent training Google DialogFlowCX, Microsoft Luis, and Amazon Lex are almost identical. They all train by providing sentences examples and are very easy to train.

Where they differ is in entity matching. Amazon Lex can capture entities by exact matching keywords. Users can define their custom entities and synonyms. Lex also features matching based on a pattern that matches words in specified parts of a sentence, for example, intent "I want a pizza with {toppings}."

Google DialogflowCX extends standard keyword matching by adding RegExp matching and fuzzy matching. Fuzzy matching is especially useful for entities that contain multi-word values. ("Entities | Dialogflow CX | Google Cloud," n.d.).Where classic entity matching requires an exact wording match, fuzzy matching can match values with a different sequence of words. For example, an entity trained using "small red ball" will match "ball red small" ("Fuzzy matching | Dialogflow CX | Google Cloud," n.d.).

Microsoft Luis is the most advanced out of these three providers. It has as well as Google DialogFlowCX regex matching of entities, but on top, it comes with features such as entity pattern matching and machine-learned entities. In pattern matching, users predefine where an entity should occur in text for example, "Where can I find {bookName}?" will then match in the sentence "Where can I find The Great Gatsby?" entity "The Great Gatsby" (aahill, n.d.). Machine learning entities can be annotated in sentences that are used for intent training based on these annotations, Luis learns what the entity is and where it can be found

(aahill, n.d.). Entities can then be also broken down into detailed structures such as order entity can have sub-entities size and quantity.

Overall winner in this category is Microsoft Luis because of the most advanced Entity matching based on machine learning. Amazon Lex and Google DialogFlowCX then tie on second place.

Google DialogFlowCX	Microsoft Luis	Amazon Lex
8	10	8

2.4.2 Automated testing

In NLU models, newly added utterances and intents retrain the whole model, which can break already working intents. It is important to have automated testing available to check how the retrained model works.

Amazon Lex does not offer automated testing out of the box it only provides a console where testing can be performed manually. There is, however solution from the Amazon team via which it is possible to automatically test end-to-end scenarios. (“Using a test framework to design better experiences with Amazon Lex,” 2020)

Google DialogFlowCX provides a simulator that is a built-in test scenario maker called simulator (“Test cases | Dialogflow CX | Google Cloud,” n.d.). Users can chat with the bot and verify if the conversation flow is correct by checking the bot’s answers, intents, and entities in each conversation step. The whole conversation then can be saved and later triggered to run automatically and evaluate results. Google offers CI/CD integration as well, but it is in the preview stage currently(“Continuous tests and deployment | Dialogflow CX,” n.d.)

Microsoft Luis offers batch testing for intents and entities unit testing (aahill, n.d.). Batches can be created via text editor or imported in JSON format. Batches can then be triggered, and results are displayed in a 2x2 confusion matrix. It also displays overall precision, recall, and F-measure. It can be further broken down into each utterance's results. Microsoft Luis testing can be integrated into CI/CD pipelines (aahill, n.d.).

The best service in automated testing is Microsoft Luis because of advanced analytics of testing and CI/CD integration which is not in the preview stage. In second place is Google DialogFlowCX, and in third Amazon Lex.

Google DialogFlowCX	Microsoft Luis	Amazon Lex
9	10	5

2.4.3 Language support

Amazon Lex can process text in 12 different languages and locales (“Languages Supported in Amazon Lex - Amazon Lex,” n.d.).

Google DialogFlowCx supports 56 languages and locales (“Language reference | Dialogflow CX,” n.d.). 24 of them are still in preview stage.

Microsoft Luis supports 19 languages and locales (aahill, n.d.). Only 1 is in preview.

Google DialogFlowCX	Microsoft Luis	Amazon Lex
8	6	5

2.4.4 Pricing

All services are usage based priced.

Amazon Lex costs $0.004 per speech request which is 4$ per 1000 request.(“Amazon Lex Pricing - Amazon Web Services,” n.d.)

Google DialogFlowCX costs $0.06 per minute for audio input and output and $0.007 per text request which is 7$ per 1000 requests. (“Pricing | Dialogflow | Google Cloud,” n.d.)

Microsoft Luis costs $5.50 per 1,000 prediction transactions for speech requests.(“Pricing - Language Understanding | Microsoft Azure,” n.d.)

Google DialogFlowCX	Microsoft Luis	Amazon Lex
5	7	9

2.4.5 Overall results

Overall winner is Microsoft Luis. In second place is Google DialogFlowCX and in last place is Amazon Luis.

	NLU Capabilities	Automated testing	Language support	Pricing	Overall score
Google DialogFlowCX	8	9	8	5	30
Microsoft Luis	10	10	6	7	34
Amazon Lex	8	5	5	9	27

2.5 Conclusion

Although Amazon's service Lex is last in the previous comparison, Amazon is still the only provider who has call center solution available out of the box. Since out of all services listed, it is easiest to integrate Amazon Lex with Amazon Connect, I would recommend using Amazon Lex as NLU provider for our contact centres.

3 Architecture

In this chapter high level architecture overview is explained together with simple diagrams.

3.1 Content ingestion from S3 to Wisdom

User story is following: Amazon Connect Wisdom suggests articles from FaQ files which will be uploaded in Amazon S3 bucket.

By default, Amazon Wisdom does not support this feature. However, it is possible to achieve it with Amazon Lambda function. Lambda function can be set up in a way that it is triggered when file is uploaded into specified S3 bucket. Lambda function then uploads file contents directly to Amazon Connect Wisdom knowledge base.

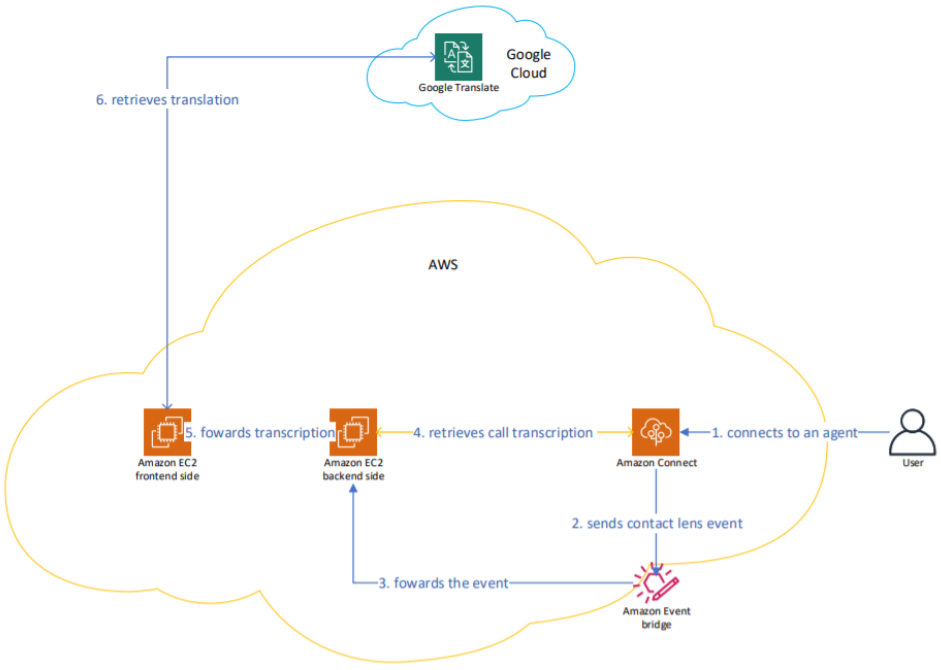
3.2 Call transcription with translation

Task is following: Display real-time call transcription and translate it to different language.

The solution I have produced is the following. Use Amazon Contact Lens (further mentioned as Contact Lens) API to receive call transcription. But to get transcription contact ID must

be provided as parameter. Contact ID is generated by Contact Lens when the user is connected to an agent. Amazon EventBridge can be triggered by events and there is an existing template for Contact Lens events. Amazon EventBridge forwards these events to specific HTTPS (HyperText Transfer Protocol Secure) API endpoint. This endpoint is exposed by NodeJS server running in EC2 Ubuntu instance, which has www domain registered in DNS (Domain Name Server). This NodeJS API retrieves contact ID from Contact Lens event and can call Contact Lens API which returns call transcription. After that the transcription is forwarded to front-end NodeJS server which is running in the same EC2 instance. Front-end server calls Google Translate API and receives translated transcription.

Simplified architecture diagram is below. Note that steps from 4 to 6 are being repeated in loop until user gets disconnected from an agent.



4 Infrastructure

This chapter describes details of HTTPS API implementation. It follows architecture which is described in chapter 3.2.

4.1 Technology overview

List with brief description

- **EC2** - "Amazon Elastic Compute Cloud (Amazon EC2) is a web service that provides secure, resizable compute capacity in the cloud." ("Amazon EC2," n.d., p. 2)
- **PuTTY** – "PuTTY is an SSH and telnet client, developed originally by Simon Tatham for the Windows platform. PuTTY is open source software that is available with source code and is developed and supported by a group of volunteers." ("Download PuTTY - a free SSH and telnet client for Windows," n.d.)
- **NodeJS** – "As an asynchronous event-driven JavaScript runtime, Node.js is designed to build scalable network applications." (Node.js, n.d.)
- **NGINX** - "NGINX is a high-performance, highly scalable, highly available web server, reverse proxy server, and web accelerator (combining the features of an HTTP load balancer, content cache, and more)." ("What is NGINX?," n.d.)
- **Certbot** - "Certbot is a free, open source software tool for automatically using Let's Encrypt certificates on manually-administrated websites to enable HTTPS."
- **GIT** - "Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency."
- **Amazon Route 53 – "**Amazon Route 53 is a highly available and scalable cloud Domain Name System (DNS) web service" (https://aws.amazon.com/route53/)

Simple overview of usage

- **EC2** for ubuntu server hosting
- **PuTTy** to access EC2 ubuntu server using SSH
- **NodeJS** to create http webservice

- **NGINX** for https and reverse proxy set up so traffic from HTTPS is passed HTTP ports which NodeJS is using
- **Certbot** for SSL configuration
- **GIT** for code version control and transfer between local and development environment
- **Amazon Route 53** for .com domain registering

4.2 Server Hosting

I have used Amazon EC2 to create NodeJS server, which can expose HTTPS API endpoint.

During creation, I have kept all settings by default except for the operating system, custom network security settings, and certificate authentication, which is suggested during setup.

As an operating system, I have used Ubuntu 18.04.

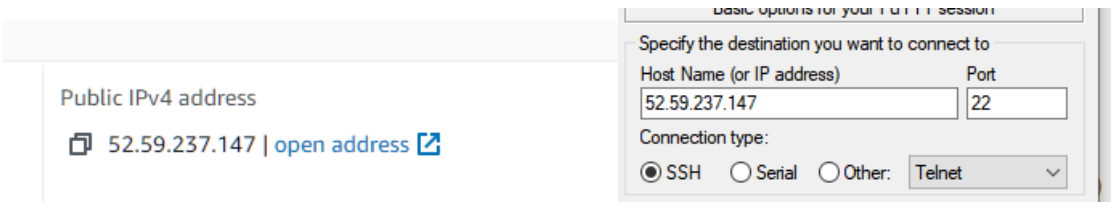
I have used already existing network security policies, which were created by Siemens IT (INFORMATION TECHNOLOGY). I have opened port 22 in a security system to connect via SSH.

4.3 Access to remote server

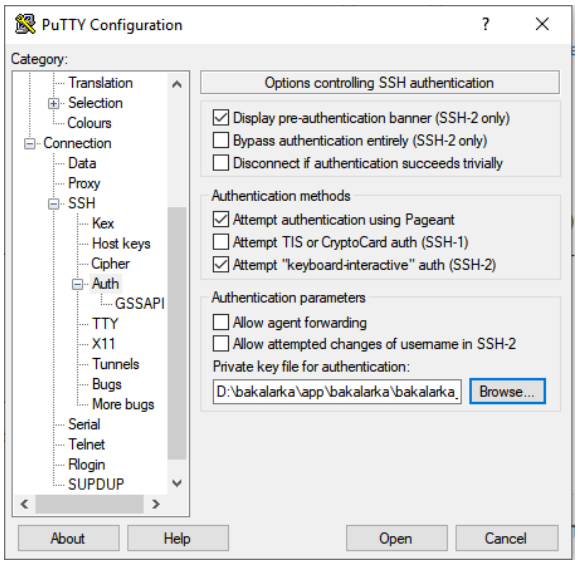
I have used PuTTy to connect remotely to created EC2 instance.

In Putty I had to configure server IP (International Protection) address and certificate authentication.

Public IP address can be found in AWS EC2 admin menu. Port number is 22 which is default SSH and firewall in AWS EC2 firewall to accept connection from my IP address.



I have configured PuTTy so that it uses private key, which has been created during EC2 set up, as authentication. This is done in Connection/SSH/Auth section. After selecting private key passphrase needs to be entered.



4.4 API endpoint set up

This chapter describes steps I had to do to establish HTTPS API endpoint.

4.4.1 NodeJS installation

Installation of NodeJS on Ubuntu 18.04 is quite simple. Entire process is to execute two commands (see picture...) .I have installed version 16.03 which was the latest stable version.

```
# Using Ubuntu
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs
```

4.4.2 Domain registration

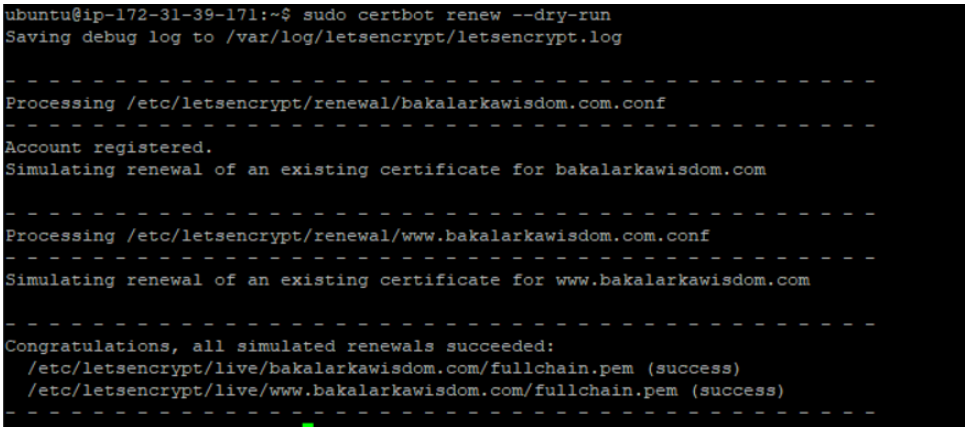
I have used Amazon service called Route 53 for domain registration. Registering domain via Route53 is an intuitive process and takes only a few clicks so I don't see any point describing it.

4.4.3 HTTPS configuration

To set up https I have used Certbot software (https://certbot.eff.org/).

Installation process:

1. Install snapd following official guide (https://snapcraft.io/docs/installing-snap-on-ubuntu)
    a. Run command “sudo apt update”
    b. Run command “sudo apt install snapd”
2. Install Certbot via command “sudo snap install --classic certbot”
3. Install https certificate and adjust nginx config automatically by running “sudo certbot --nginx”
    a. Several options need to be filled out. Most important is domain name which has to match server DNS domain name otherwise certificate will not be accepted by modern browsers.
4. Test if https is working correctly by executing test certificate renewal command “sudo certbot renew --dry-run”



4.4.4 Reverse proxy

NodeJS server is not allowed to listen on port number 443 by default. The solution is to use a different port and set up a reverse proxy to forward traffic from 443 (default HTTPS port) there.

For this, I have used NGINX.

I have installed NGINX using the command “Sudo apt-get install nginx” command.

I had to set up a reverse proxy to three different http ports.

- http://localhost:9080 - used by client side.
- http://localhost:9000 - used by API endpoint which receives messages from Amazon EventBridge

- http://localhost:9100 – used by server side to establish socket.io communication
-

I have done following steps to set up reverse proxies to mentioned ports:

1) Opened NGINX config file in nano

```
$ sudo nano /etc/nginx/sites-enabled/default
```

2) Added following lines inside Server { }

a.
```
    location / {
            # First attempt to serve request as file, then
            # as directory, then fall back to displaying a 404.
            #try_files $uri $uri/ =404;
           proxy_pass http://localhost:9080;
           proxy_http_version 1.1;
           proxy_set_header Upgrade $http_upgrade;
           proxy_set_header Connection 'upgrade';
           proxy_set_header Host $host;
           proxy_cache_bypass $http_upgrade;
    }
```

b.
```
       location /api {
          proxy_pass http://localhost:9000;
              proxy_http_version 1.1;
              proxy_set_header Upgrade $http_upgrade;
              proxy_set_header Connection 'upgrade';
              proxy_set_header Host $host;
              proxy_cache_bypass $http_upgrade;
       }
```

c.
```
location ~* \.io {
  proxy_set_header X-Real-IP $remote_addr;
  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  proxy_set_header Host $http_host;
  proxy_set_header X-NginX-Proxy false;

  proxy_pass http://localhost:9100;
  proxy_redirect off;

  proxy_http_version 1.1;
  proxy_set_header Upgrade $http_upgrade;
  proxy_set_header Connection "upgrade";
}
```

3) Closed and saved config file by pressing CTTR+X and then confirm save by pressing Y.

4.5 Cloning code and starting NodeJS server

I first needed to clone my repository first. To do that, I have installed git using apt install.

Then I have executed following steps.

1) Clone repository
    a. Command “git clone url”
2) Install project dependencies
    a. Command “npm install”
3) Compile code and start NodeJS
    a. Commands “npm run build” and “npm start”

5 Cloud services

This chapter focuses on how cloud services, which I have used in this project, have been set up.

I will not go into details on how to set up Google translation API in google cloud console, as I have already used an existing one.

5.1 Amazon Connect

AWS Connect is a cloud service whose goal is to "provide superior customer service at a lower cost" ("Cloud Contact Center – Amazon Connect – Amazon Web Services," n.d.).

It is remarkably simple to set up a cloud contact center in Amazon Connect. During the setup process, I left all configurations on default values.

After deployment had finished, I selected phone numbers from a toll-free list and created a custom contact flow that has its own subchapter.

5.1.1 Terms definition

For better understanding, I will briefly introduce concepts of contact center that are relevant for further understanding of this thesis.

- Admin is a person managing conversation rules, assigning rights to agents.
- Agent is a person answering users calls.
- User is a person calling to AWS Connect number.
- Contact flow – "Contact flow defines the customer experience with your contact center from start to finish. Amazon Connect includes a set of default contact flows so you can quickly set up and run a contact center. However, you may want to create custom contact flows for your specific scenario." (https://docs.aws.amazon.com/connect/latest/adminguide/connect-contact-flows.html)
- Queue – If no agent is available users must wait in queue.

5.1.2 Amazon Wisdom

Amazon Connect provides Amazon Wisdom integration with built in connectors to Salesforce and ServiceNow

I needed to use S3 as a data source, which is not available, but integration can be manually created.

I have followed steps from Amazon blog post.

AWS Wisdom Assistant – AI based solution that receives text input and can suggest best matching sections from articles which are stored in AWS Wisdom knowledge base.

Firstly, I have created AWS Wisdom and integrated it with mine AWS Connect instance.

I had to Commands

1) Create AWS Wisdom Knowledgebase
2) Create AWS Wisdom Assistant
3) I have integrated created assistant with knowledgebase.
4) I have integrated assistant with my instance of AWS Connect

Secondly, I have set up data ingestion from Amazon S3 bucket to AWS Wisdom knowledgebase.

I had to

1) Create AWS Lambda function.
2) Set up trigger which executes our Lamba function when someone uploads file to specified S3 bucket. This trigger sends event, which has all data required for file upload to Wisdom knowledgebase.

3) Set up lambda code, which uploads file contents to Wisdom knowledgebase. I have reused code from blog post, which is mentioned above, and modified necessary fields.

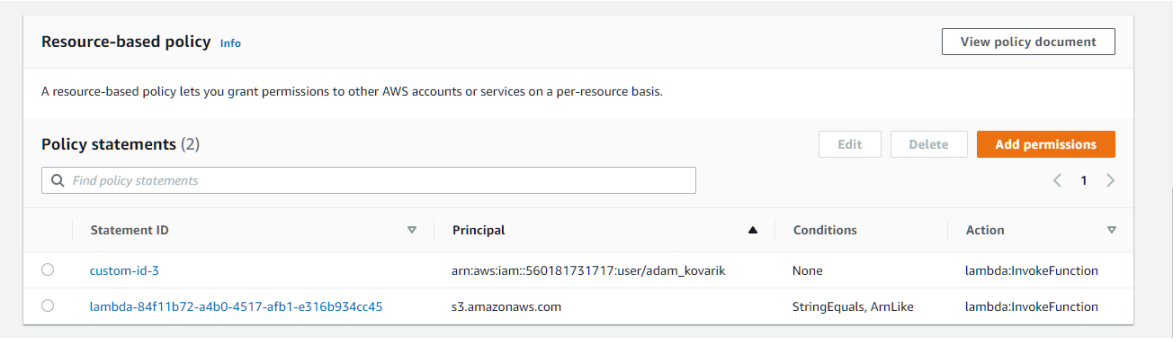
```
const aws = require('aws-sdk');
const axios = require('axios')
const s3 = new aws.S3({ apiVersion: '2006-03-01' });

exports.handler = async (event, context) => {
    // Get the object from the event and show its content type
    const bucket = event.Records[0].s3.bucket.name;
    const key = decodeURIComponent(event.Records[0].s3.object.key.replace(/\+/g, ' '));
    const params = {
        Bucket: bucket,
        Key: key,
    };
    const knowledgeBaseId = "40c7af88-c937-4b64-a6ad-b00281c19cc0";
    try {
        const s3Object = await s3.getObject(params).promise();
        console.log("s3Object", s3Object)
        var client = new aws.Wisdom();
        const uploadDetails = await client.startContentUpload({
            "knowledgeBaseId": knowledgeBaseId,
            "contentType": s3Object.ContentType,

        }).promise();
        console.log(uploadDetails)
        await axios.put(uploadDetails.url, s3Object.Body, {
            headers: uploadDetails.headersToInclude
        });
        const searchResponse = await client.searchContent({
            "knowledgeBaseId": knowledgeBaseId,
            "searchExpression": {
                "filters": [{
                    "field": "name",
                    "operator": "equals",
                    "value": params.Key
                }]
            }
        }).promise();
        if (searchResponse.contentSummaries.length === 0) {
            await client.createContent({
                "knowledgeBaseId": knowledgeBaseId,
                "name": params.Key,
                "uploadId": uploadDetails.uploadId,
                "metadata": {
                    "s3Version": s3Object.VersionId
                }
            }).promise();
        } else {
            await client.updateContent({
                "knowledgeBaseId": knowledgeBaseId,
                "contentId": searchResponse.contentSummaries[0].contentId,
                "revisionId": searchResponse.contentSummaries[0].revisionId,
                "uploadId": uploadDetails.uploadId,
                "metadata": {
                    "s3Version": s3Object.VersionId
                }
            }).promise();
        }
        console.log('searchResponse', JSON.stringify(searchResponse))
        console.log('CONTENT TYPE:', s3Object.ContentType);
        return s3Object.Body;

    } catch (err) {
        console.log(err);
        console.log(err.message);
        throw new Error(err.message);
    }
};
```

4) Configure Lambda's permissions so it can access desired S3 storage and Wisdom knowledgebase. I have already used existing policies.

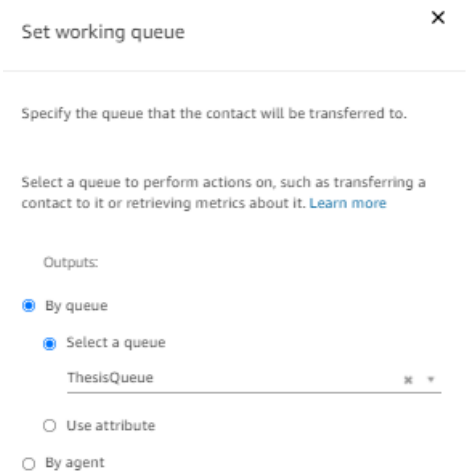


5.1.3 Contact flow

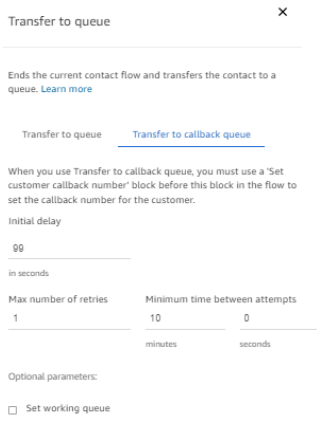
Contact flow in AWS Connect is where all routing logic and business rules lie. It can be configured in a low code browser editor, and if needed, lambda function can be executed. By itself, it provides quite an intensive set of tools such as built-in integration of text-to-speech, number prompts, integration with AWS Lex.

Below is described how I created and set up the contact flow for this project.

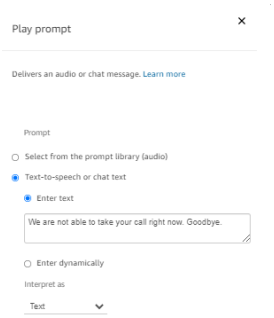
1) I have enabled integration with contact lens and configured its behaviour so that it records both agent's and user's voice, use ML-based speech analytics such as call transcript and sentiment analysis.
2) I have enabled integration with AWS wisdom assistant.
3) I have configured how and into which queue user should be routed. I have configured it in a way, that the user is always transferred to Thesis Queue.



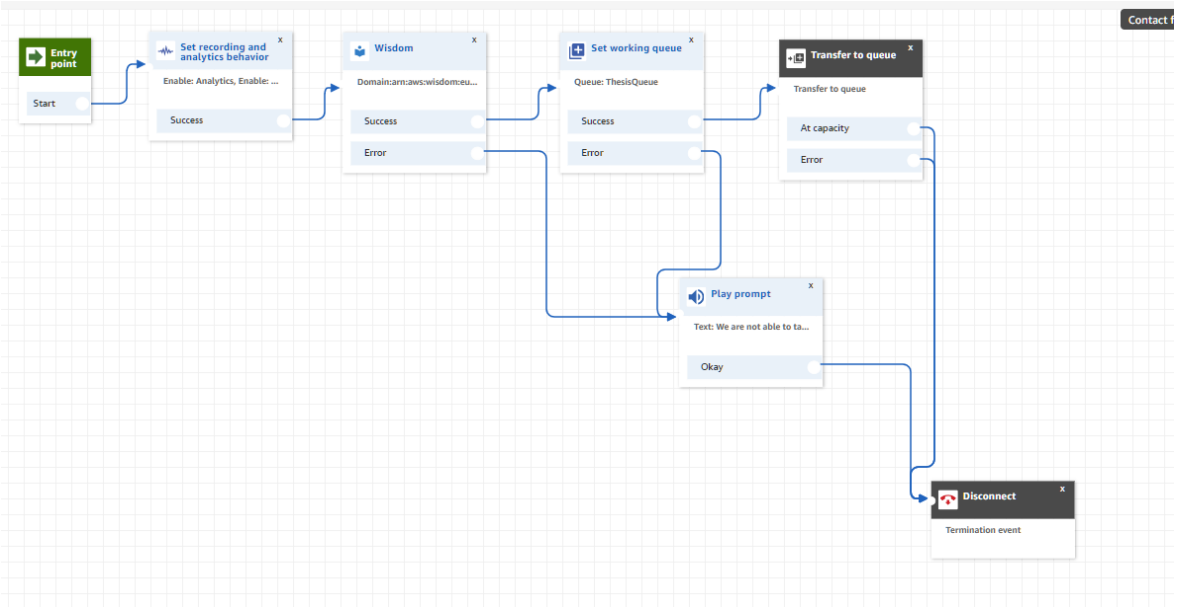
4) Actual transfer to queue based on rules evaluation from previous step. I kept here default configuration.



In case there is an error during the previous steps, prompt is then played to the customer and afterwards he will be disconnected.



Whole contact flow configuration is shown in picture below.



5.2 Amazon EventBridge

"Amazon EventBridge is a serverless event bus that makes it easier to build event-driven applications at scale using events generated from your applications, integrated Software-as-a-Service (SaaS) applications, and AWS services." (https://aws.amazon.com/eventbridge/)

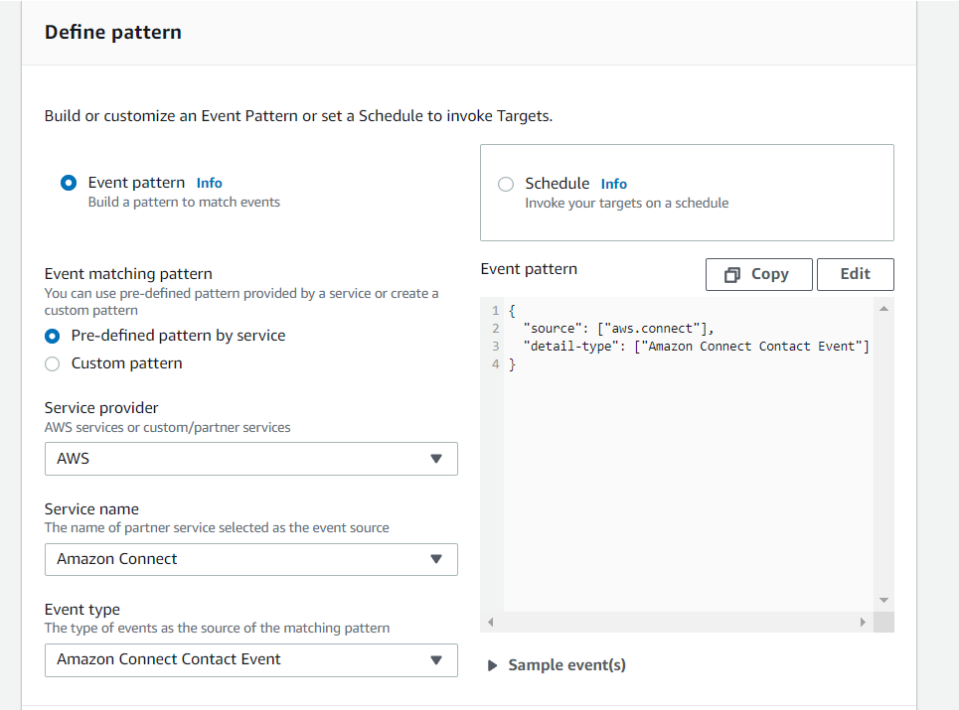
To get the real-time transcription of the call from Amazon Connect API, there needs to be passed caller ID, which is generated when a user connects to an agent. I have found out that caller ID is passed in an event called "CONNECTED_TO_AGENT".

Sample here

I have used Amazon EventBridge, which can be triggered by Amazon Contact Lens events and even has this integration built-in. I just needed to select a predefined pattern and set up an API destination to which these events should be forwarded.

5.2.1 Define pattern

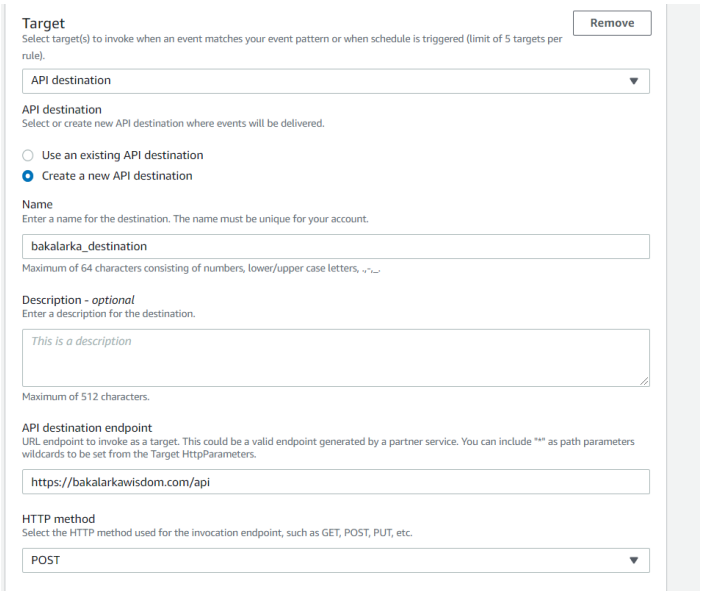
There are two options how to trigger rule. It can be either by schedule or event. In this case I needed an event. Events can be selected from template, or custom defined. Since Contact Lens Events are available in templates, I did not need to configure it myself.



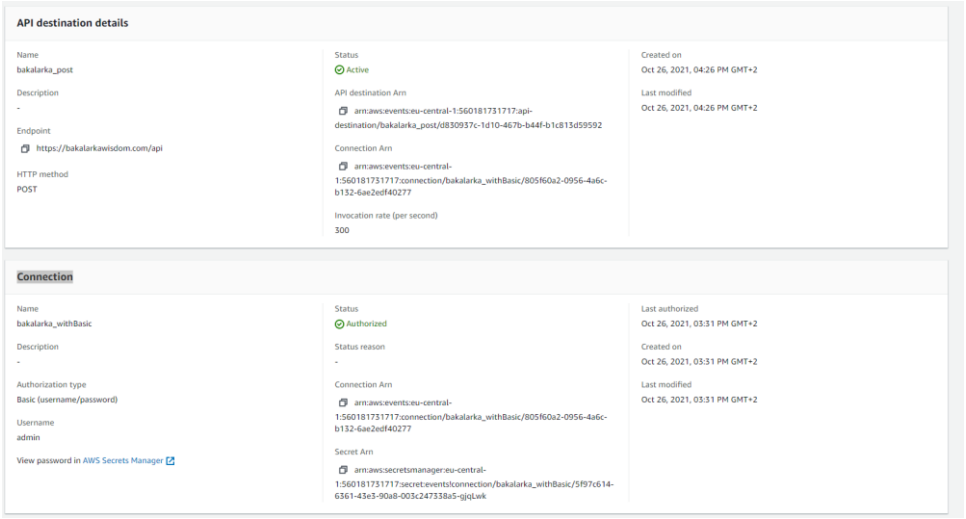
I needed to configure my https endpoint as the target. Select API destination from list

5.2.2 API destination

1. To create API destination, I had to define the name, endpoint, and HTTP method.



2. Then I had to define the authorization type, which is in my case Basic and enter username with password.



6 Server code

To achieve goal of this proof of concept, application must fulfil following requirements.

- Agents can answer calls
- Agents can see recommended articles based on customer queries.
- Agents can see real time transcribe of call and its translation to desired language.
- Agents can see customers overall sentiment.

Note: I have built this as a proof of concept, so I have assumed a scenario that only one Agent is active and only one customer is calling. Credentials should never be stored in code directly. But since I am doing PoC and will change all credentials afterward and I have decided not to complicate the solution with credentials management. Also, there is no exception handling which should always be implemented in production code.

6.1 Local development environment set up

My local development environment is running on Windows as an operation system. Following chapters assume Windows as an operating system.

6.1.1 Visual Studio Code set up

I have downloaded and installed the Visual Studio Code from official website (https://code.visualstudio.com/download) .

I have used many extensions, but most useful for me have been GitLens — Git supercharged and Prettier ESLint.

GitLens simplify working with commit history and checking individual file history differences.

Prettier ESLint I have used for code formatting and syntax check. I have modified default settings by creating file .eslintrc.json

```
{
    "env": {
        "browser": true,
        "es6": true
    },
    "extends": [
        "eslint:recommended",
        "plugin:@typescript-eslint/eslint-recommended"
    ],
    "globals": {
        "Atomics": "readonly",
        "SharedArrayBuffer": "readonly"
    },
    "parser": "@typescript-eslint/parser",
    "parserOptions": {
        "ecmaVersion": 2018,
        "sourceType": "module"
    },
    "plugins": [
        "@typescript-eslint"
    ],
    "rules": {
    },
    "ignorePatterns": ["node_modules/*.js"]
}
```

6.1.2 NodeJS set up

I have downloaded and installed NodeJS latest stable version 16.13.0. All decencies are specific to server / client side, and they are mentioned in according to sections.

6.1.3 Git installation

To work with git repository locally I have used Git I have download and installed Git from Windows from here… (https://git-scm.com/download/win).

6.2 Git repository set up

“Git is a free and open-source distributed version control system designed to handle everything from small to huge projects with speed and efficiency.” (https://git-scm.com/)

I have used git as my version control manager in this project mainly because I am familiar with the software.

I have created a repository in code.siemens.com, which is based on an open-source version of GitLab and is the official Siemens DevOps platform.

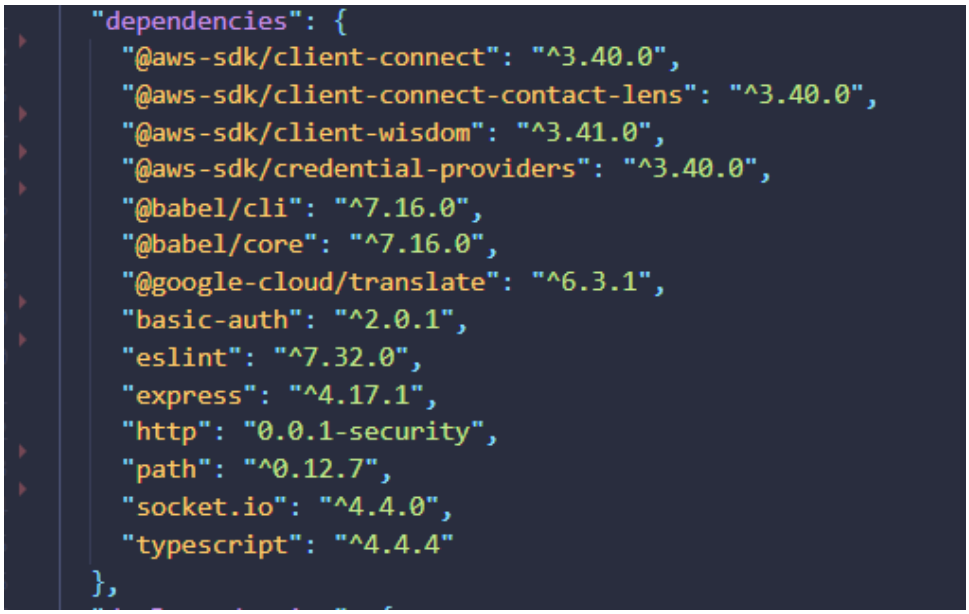
I have used SSH connection with certificate authentication for working with the origin repository. Steps how to set it up are well described in official GitLab documentation (https://docs.gitlab.com/ee/ssh/)

6.3 Backend-side

In this chapter, I will briefly describe backend side code with samples of code. I have tried to layer the coder well to make code better readable and maintainable in the future.

6.3.1 Environment set up

Below are all libraries with version that I have used in backend-side code. All libraries are publicly available in www.npmjs.com and thus can be installed simply by using the command “npm install”.



I will describe important libraries below.

- Typescript
    - I have decided to use Typescript in server side since I find it to be more mature language then plain JavaScript. This library compiles Typescript code to plain JavaScript.
- Express (https://www.npmjs.com/package/express)
    - I have used express library to create simple http server with an API endpoint that can receive post calls.
    - Installed from npm .
- Basic-auth (https://www.npmjs.com/package/basic-auth)
    - I have used this library to secure Express API endpoint with basic authentication.
- @aws-sdk/client-connect-contact-lens
    - I have used this library to invoke Amazon Connect APIs.
- @aws-sdk/credential-providers
    - Used for loading AWS credentials which are stored as environment variables.
- Socket.io (https://www.npmjs.com/package/socket.io)
    - I have used it to simplify WebSocket communication between backend-client.

6.3.2 Overview

Backend-side code should do following:

- Secure and define API endpoint to which Amazon Event Bridge can forward events.
- Execute Amazon Contact Lens API to receive call transcribe.
- Forward new transcription from Contact lens to the front end.

Below is the content of the whole main / index file. It first sets environment variables which are later used by the Contact Lens library. Then it defines the port for socket.io server and express server. Both servers are then created and listened to on mentioned ports.

```
(async () => {

  Util.setEnviromentVariables();

  const portIo = 9100;
  const portExpress = 9000;
  const expressSimple = new ExpressSimple();
  const io = IoSimple.createIo(portIo);

  expressSimple.createSeverAndListen(io, portExpress);
})();
```

6.3.3 API details

Goals

- Secure and create API endpoint to which Amazon Event Bridge can forward events.
- Endpoint should manage class ConnectLensSimple and not work with Contact lens API directly.

I have used the express library to simplify the creation of the API endpoint, which I have wrapped in class ExpressSimple. It has a single public method, createSeverAndListen, which is executed from the main / index fi

```
public createSeverAndListen = (io, portExpress) => {
  this.setupAPIEndpoint(io);
  this.app.listen(portExpress, () =>
    console.log(`Example app listening on port ${portExpress}!`)
  );
};
```

Method createBasicAuth is middleware that is passed as a parameter into express method "post." It parses the request body using the basic-auth library. If provided username and password are valid, then the code inside of the "post" method is executed. Otherwise, error code 401 is sent to the requestor.

```
private createBasicAuth = (req, res, next) => {
  const user = basicAuth(req);
  if (!user || !user.name || !user.pass) {
    res.set("WWW-Authenticate", "Basic realm=Authorization Required");
    res.sendStatus(401);
    return;
  }
  if (user.name === "admin" && user.pass === "BakalarkaTest123!") {
    next();
  } else {
    res.set("WWW-Authenticate", "Basic realm=Authorization Required");
    res.sendStatus(401);
    return;
  }
};
```

Method setUpAPIEndpoint is creating API endpoint and defining logic that should be executed. It uses the previously defined method createbasicAuth for authentification.

Simplified logic of code which is executed after successful request can be defined as follows: If the user is connected to an agent, get his contact ID and start forwarding transcribed text from Contact Lens to the front end. If the user gets disconnected from the call, don't forward anything.

```
private setupAPIEndpoint = io => {
  this.app.post("/api", this.createBasicAuth, async function(req, res) {
    try {
      const contactLens = new ContactLensSimple();
      const eventBody = req.body;
      const eventType = ContactLensSimple.getEventContactLensEventType(
        eventBody
      );
      if (eventType === EventType.CONNECTED_TO_AGENT) {
        contactLens.setUserConnected(true);
        const contactId = ContactLensSimple.getContactId(eventBody);
        contactLens.getCallTranscriptionAndForwardToClient(contactId, io);
      }
      if (eventType === EventType.DISCONNECTED) {
        contactLens.setUserConnected(false);
      }
      res.send("success");
      res.end();
    } catch (e) {
      console.log(`/api error: ${JSON.stringify(e)}`);
    }
  });
```

6.3.4 Contact lens transcription retrieval

I have created class called ContactLensSimple which methods wrap @aws-sdk/client-connect-contact-lens library. Its public methods are used by ExpressSimple class.

In the constructor, the class ConnectContactLensClient is initialized with credentials that are loaded from environment variables.

```
export class ContactLensSimple {
  private lens: ContactLensfrom.ConnectContactLensClient;
  constructor() {
    const region = "eu-central-1";
    this.lens = new ContactLensfrom.ConnectContactLensClient({
      credentials: fromEnv(),
      region
    });
  }
```

GetContactId method returns contact ID from event body.

```
public static getContactId = (eventBody: IConnectEvent) => {
  console.log(`user ID: ${eventBody.detail.contactId}`);
  return eventBody.detail.contactId;
};
```

Method getTranscribe retrieves transcription from Contact lens and if there is new text, it returns it otherwise it returns undefined. I have also added logging of errors.

```
private async getTranscribe(io: any, contactId: string) {
  try{
    const params: ListRealtimeContactAnalysisSegmentsCommandInput = {
    InstanceId: "5f4160be-c97f-403c-b21d-b109162b8c73",
    ContactId: contactId
    };
    const command = new ContactLens.ListRealtimeContactAnalysisSegmentsCommand(
    params
    );
    const response = await this.lens.send(command);
    const userTranscribe =
    response.Segments[response.Segments.length - 1] ? response.Segments[response.Segments.length - 1].Transcript.Content : "";
    if (!this.previousSentTranscribe && userTranscribe){
        this.previousSentTranscribe = userTranscribe;
        return userTranscribe;
    } else if(this.previousSentTranscribe !== userTranscribe && userTranscribe) {
        this.previousSentTranscribe = userTranscribe;
        return userTranscribe;
    } else {
        //same text as the last one
        return undefined;
    }
  } catch (e) {
    console.log(e);
  }
}
```

GetCallTranscriptionAndForwardToClient forwards transcripted text every two seconds until user gets disconnected form an agent.

```
public getCallTranscriptionAndForwardToClient = async (
  contactId: string,
  io: any
) => {
  while (this.isUserConnected) {
    const transcribe = await this.getTranscribe(io, contactId);
    if(transcribe){
      io.emit("customerTranscript", transcribe);
    }
    await Util.wait(2000);
  }
};
```

6.3.5 Socket.io

Class IoSimple has single static public method create Io that creates server which listens on port that is passed as parameter.

```
You, an hour ago | 1 author (You)
export class IoSimple {
  constructor() {}

  public static createIo = portIo => {
    const httpServer = createServer();
    const io = new Server(httpServer, {
      cors: {
        origin: "*"
      }
    });
    httpServer.listen(portIo, () =>
      console.log(`IO listening on port: ${portIo}`)
    );
    return io;
  };
}
```

6.4 Client-side code

In this chapter, I will briefly describe the client-side together with samples of code. For the agent front-end web app, I have decided to use react library, which I find useful mainly because of its modularity, simple state management, and HTML component manipulation. Client-side code is written in JavaScript.

6.4.1 Environment set up

With command “npx create-react-app my-app" I have created simple react project with all required dependencies installed. I have then installed few additional ones to expand functionality. I will shortly describe the use of important libraries, which I have installed additionally, below.

- @google-cloud/translate
    - Library which simplifies communication with Google Translate API.
- amazon-connect-streams
    - Renders Amazon Connect iframes.
- react-bootstrap
    - Provides built-in DOM (Document Object Model) layouts and components with predefined CSS styles.
- socket.io-client
    - Creates client-side connection to server’s socket.io server

```
"dependencies": {
  "@google-cloud/translate": "^6.3.1",
  "@testing-library/jest-dom": "^5.11.4",
  "@testing-library/react": "^11.1.0",
  "@testing-library/user-event": "^12.1.10",
  "amazon-connect-streams": "git+https://github.com/amazon-connect/amazon-connect-streams.git",
  "bootstrap": "^5.1.3",
  "react": "^17.0.2",
  "react-bootstrap": "^2.0.2",
  "react-dom": "^17.0.2",
  "react-scripts": "4.0.3",
  "socket.io-client": "^4.4.0",
  "web-vitals": "^2.1.2"
},
```

6.4.2 Overview

What must be implemented:

1. Call management (agent can pick up calls, manage his availability, etc.)
2. Wisdom article suggestions.
3. Display user speech transcription and it's translation.
4. Option to select translation language.

In chapters below I will describe how this has been implemented.

6.4.3 Call management

With the use of amazon-connect-streams connect stream, I only needed to invoke the initCCP method inside the useEffect function and return div in which Amazon Call management iframe renders.

```
const CCP_URL = "https://bakalarka-pokus4.my.connect.aws/connect/ccp-v2";
const REGION = "eu-central-1";

function Ccp() {
  useEffect(() => {
    const container = document.getElementById("ccp");
    /*global someFunction, a*/
    /*eslint no-undef: "warn"*/

    connect.core.initCCP(container, {
      ccpUrl: CCP_URL,
      loginPopup: true,
      loginPopupAutoClose: true,
      region: REGION,
      softphone: {
        allowFramedSoftphone: true
      }
    });
  }, []);

  return <div id="ccp" style={{ width: "320px", height: "500px" }}></div>;
}
```

6.4.4 Wisdom article suggestion

Again, quite simple integration. Invoke method initApp from amazon-connect-streams inside useEffect function and return div in which Amazon Wisdom iframe renders.

```
const connectUrl = "https://bakalarka-pokus4.my.connect.aws/connect"

function WisdomIframe() {
  useEffect(() => {
    const container = document.getElementById("wisdomIframe");
    /*global someFunction, a*/
    /*eslint no-undef: "warn"*/
    connect.agentApp.initApp("wisdom","wisdomIframe",connectUrl + "/wisdom-v2/");
  }, []);

  return <Col id="wisdomIframe" style={{ width: "400px", height: "600px" }}></Col>;
}
```

6.4.5 Transcription with translation

This part of the code is responsible for displaying real-time transcription and translation, plus an option for the user to select desired translation language. To implement this functionality, I needed to use multiple libraries.

I have created the class BootstrapTextarea, which extends React.Component class, which was built in a state management. That is useful for keeping track of selected language, displayed text, and translated text.

- I n constructor, I have initialized Translate class from @google-cloud/translate library and io class from socket.io-client.

```
constructor() {
  super();
  this.state = {
    lang: null,
    userText: "",
    translatedText: ""
  };
  const options = {
    key: "AIzaSyCb_8QJV4NC3JHD-bGlyxDF0peXzqSc3XA"
  };
  this.translator = new GoogleTranslator.v2.Translate(options);
  this.handleInputChange = this.handleInputChange.bind(this);
  this.socket = io("https://bakalarkawisdom.com");
```

Method componentDidMount is inherited from React.Component and it “is invoked immediately after a component is mounted (inserted into the tree)”. (https://reactjs.org/docs/react-component.html#componentdidmount)

After that it is updating state with transcription that is sent from server side.

```
componentDidMount() {
  this.socket.on("customerTranscript", customerTranscriptText => {
    const state = this.state;
    state.userText = state.userText + customerTranscriptText;
    this.setState(state)
  })
}
```

Method translate text accepts text and translation language as parameters. It then invokes the translate method and updates state property with translated text.

```
translateText = async (text, to = "DE") => {
  if (text) {
    if (!to) {
      to = "DE"
    }
    const translation = this.translator.translate(text, {
      from: "EN",
      to
    }).then(res => {
      const translatedText = res[0];
      const state = this.state;
      state.translatedText = translatedText;
      console.log(`tranlsated: ${translatedText}`)
      this.setState(state)
    });
  }
  return null;
};
```

Method setLanguage parses event sent by form select and updates state property.

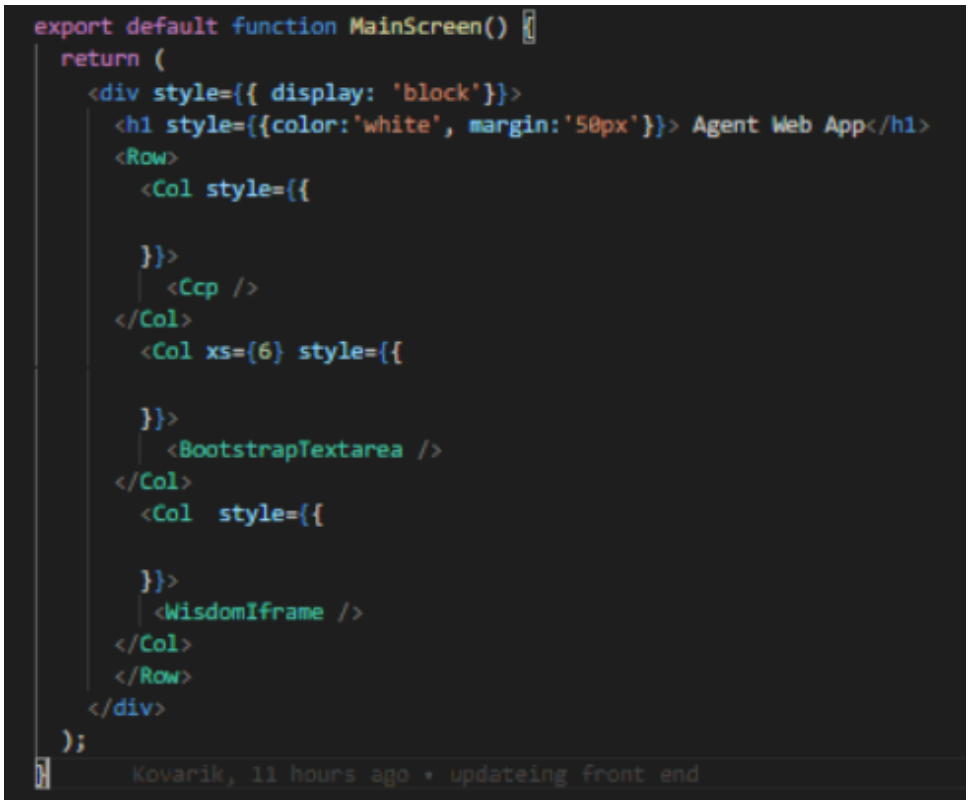
```
setLanguage(event) {
  const lang = event.target.value;
  this.setState({
    lang: lang
  });
}
```

Inherited method renders returns form with two text areas components and select component. I have used components from the react-bootstrap library. The first text area displays the current value of state—userText property. Select has three languages options. Property state.lang is updated on change with selected language, and method translate Text is invoked as well. The second text area displays the current value of property state.translatedText.

```
render() {
  return (
    <div>
      <Form>
        <Form.Group className="mb-3" controlId="exampleForm.ControlTextarea1">
          <Form.Label>Customer speech transcription</Form.Label>
          <Form.Control
            as="textarea"
            rows="3"
            name="address"
            value={this.state.userText}
          />
          <Form.Select
            aria-label="Default select example"
            onChange={e => {
              this.setState({ lang: e.target.value })
              this.translateText(this.state.userText, e.target.value)
              }
            }
          >
            <option>Select language</option>
            <option value="DE">DE</option>
            <option value="CS">CZ</option>
            <option value="PT">PT</option>
          </Form.Select>
          <Form.Control
            as="textarea"
            rows="3"
            name="address"
            value={this.state.translatedText}
          />
        </Form.Group>
      </Form>
    </div>
  );
}
```

6.4.6 Layout

Layout of web application is handled function MainScreen. With use of react-bootstrap library previously described components are put together in separate columns.


export default function MainScreen() {
return (
<div style={{ display: 'block'}}>
<h1 style={{color:'white', margin:'50px'}}> Agent Web App</h1>
<Row>
<Col style={{
}}>
<Ccp />
</Col>
<Col xs={6} style={{
}}>
<BootstrapTextarea />
</Col>
<Col style={{
}}>
<WisdomIframe />
</Col>
</Row>

);
Kovarik, 11 hours ago • updateing front end

7 Feedback

Matthias has found few security leaks. I will list the most important ones below:

- I had SSH port 22 opened to all IP addresses. It should be limited to specific ones or range at least.
- I have used non hardened version of Ubuntu OS which means it would not pass info security check. Hardened means version which is much stricter about security rules than standard version.

7.1 Cloud security

Matthias has found few security leaks. I will list the most important ones below:

- I had SSH port 22 opened to all IP addresses. It should be limited to specific ones or range at least.
- I have used non hardened version of Ubuntu OS which means it would not pass info security check. Hardened means version which is much stricter about security rules than standard version.

7.2 Ubuntu server configuration

Most interesting part of the interview was the end. After seeing whole solution Matthias proposed simpler solution that would be serverless. His proposal is following.

Instead of having https API in own server I could use service Amazon API Gateway, to which I can forward messages from Amazon EventBridge. And, in theory frontend I have created could be statically served and code logic executed in client.

7.3 Overall feedback

Most interesting part of the interview was the end. After seeing whole solution Matthias proposed simpler solution that would be serverless. His proposal is following.

Instead of having https API in own server I could use service Amazon API Gateway, to which I can forward messages from Amazon EventBridge. And, in theory frontend I have created could be statically served and code logic executed in client.

8 Conclusion

Overall, I see this PoC (Proof of concept) as successful as it fulfilled the main goal, which was utterly replacing the previously built solution, which is using Amazon Kendra, by solution that uses Amazon Wisdom instead.

However, as proposed by Matthias Schardt, I agree that this solution is unnecessarily complicated. To continue with this web app, I would suggest reusing code parts from this project and replacing the whole architecture based on ideas that are discussed in chapter 6.3.

Microsoft, n.d. Entity types - LUIS - Azure Cognitive Services [WWW Document]. Available at: https://docs.microsoft.com/en-us/azure/cognitive-services/luis/luis-concept-entity-types (13.12.2021).

Microsoft, n.d. Language support - LUIS - Azure Cognitive Services [WWW Document]. Available at: https://docs.microsoft.com/en-us/azure/cognitive-services/luis/luis-language-support (accessed 29.7.2021).

Amazon, n.d. Amazon Connect Wisdom – Amazon Web Services [WWW Document], Available at: https://aws.amazon.com/connect/wisdom/ (accessed 13.12.2021).

Amazon EC2 [WWW Document], n.d. . Amaz. Web Serv. Inc. URL https://aws.amazon.com/ec2/ (accessed 12.13.21).

Amazon, n.d. Amazon Lex Pricing - Amazon Web Services [WWW Document], Available at: https://aws.amazon.com/lex/pricing/ (accessed 13.12.2021).

Amazon [WWW Document], n.d. . Amaz. Web Serv. Inc. URL https://aws.amazon.com/ec2/ (accessed 12.13.21).

Bengio, Y., Ducharme, R., Vincent, P., Jauvin, C., 2003. A Neural Probabilistic Language Model. J. Mach. Learn. Res. 3, 1137–1155.

Microsoft, n.d. Teams Contact Center - Microsoft Teams [WWW Document]. Available at: https://docs.microsoft.com/en-us/microsoftteams/teams-contact-center (accessed 13.12.2021).

Anywhere365, n.d. Cloud Contact Center Solution | CCaaS [WWW Document], Anywhere365®. Available at: https://anywhere365.io/contact-center/ (accessed 21.7.2021).

Google, n.d. Contact Center AI [WWW Document], Google Cloud. Available at: https://cloud.google.com/solutions/contact-center (accessed 21.7.2021).

Google, n.d. Continuous tests and deployment | Dialogflow CX [WWW Document], Google Cloud. Available at: https://cloud.google.com/dialogflow/cx/docs/concept/continuous-tests?hl=cs (accessed 13.12.2021).

Putty, n.d. Download PuTTY - a free SSH and telnet client for Windows [WWW Document], Available at: https://www.putty.org/ (accessed 13.12.2021).

Google, n.d. Entities | Dialogflow CX | Google Cloud [WWW Document], Available at: https://cloud.google.com/dialogflow/cx/docs/concept/entity (accessed 13.12.2021).

Entities | Dialogflow CX [WWW Document], n.d. . Google Cloud. URL https://cloud.google.com/dialogflow/cx/docs/concept/entity?hl=cs (accessed 12.13.21).

Frome, A., Corrado, G., Shlens, J., Bengio, S., Dean, J., Ranzato, M., Mikolov, T., 2013. DeViSE: A Deep Visual-Semantic Embedding Model, in: Neural Information Processing Systems (NIPS).

Google, n.d. Fuzzy matching | Dialogflow CX | Google Cloud [WWW Document], Available at: https://cloud.google.com/dialogflow/cx/docs/concept/entity-fuzzy (accessed 13.12.2021).

INTERSPEECH 2012 Abstract: Sundermeyer et al. [WWW Document], n.d. URL https://www.isca-speech.org/archive_v0/interspeech_2012/i12_0194.html (accessed 12.13.21).

Jones, K.S., 1994. Natural Language Processing: A Historical Review, in: Zampolli, A., Calzolari, N., Palmer, M. (Eds.), Current Issues in Computational Linguistics: In Honour of Don Walker. Springer Netherlands, Dordrecht, pp. 3–16. https://doi.org/10.1007/978-0-585-35958-8_1

Joseph, S., Sedimo, K., Kaniwa, F., Hlomani, H., Letsholo, K., 2016. Natural Language Processing: A Review. Nat. Lang. Process. Rev. 6, 207–210.

Google, n.d. Language reference | Dialogflow CX [WWW Document], Google Cloud. Available at: https://cloud.google.com/dialogflow/cx/docs/reference/language (accessed 29.7.2021).

Amazon, n.d. Languages Supported in Amazon Lex - Amazon Lex [WWW Document], Available at: https://docs.aws.amazon.com/lex/latest/dg/how-it-works-language.html (accessed 29.7.2021)

Liddy, E.D., n.d. Natural Language Processing 15.

Mikolov, T., Karafiat, M., Burget, L., Cernocky, J., Khudanpur, S., n.d. Recurrent Neural Network Based Language Model 4.

Microsoft, n.d. Use active learning with knowledge base - QnA Maker - Azure Cognitive Services [WWW Document]. Available at: https://docs.microsoft.com/en-us/azure/cognitive-services/qnamaker/how-to/use-active-learning (accessed 13.12.2021).

Node.js, n.d. About [WWW Document]. Node.js. URL https://nodejs.org/en/about/ (accessed 12.13.21).

Anywhere365, n.d. Omnichannel Dialogue Cloud Platform [WWW Document], Anywhere365®. URL https://anywhere365.io/category/anywhere365-dialogue-cloud/ (accessed 21.7.2021).

Google, n.d. Pricing | Dialogflow | Google Cloud [WWW Document], Available at: https://cloud.google.com/dialogflow/pricing (accessed 13.12.2021).

Microsoft, n.d. Pricing - Language Understanding | Microsoft Azure [WWW Document], Available at: https://azure.microsoft.com/en-us/pricing/details/cognitive-services/language-understanding-intelligent-services/ (accessed 13.12.2021).

Reshamwala, A., Mishra, D., Pawar, P., 2013. REVIEW ON NATURAL LANGUAGE PROCESSING. IRACST – Eng. Sci. Technol. Int. J. ESTIJ 3, 113–116.

Amazon, n.d. Serverless Computing - AWS Lambda - Amazon Web Services [WWW Document], Available at: https://aws.amazon.com/lambda/ (accessed 13.12.2021).

Standford, n.d. SHRDLU [WWW Document]. SHRDLU. URL https://hci.stanford.edu/~winograd/shrdlu/ (accessed 13.12.2021).

Google, n.d.Test cases | Dialogflow CX | Google Cloud [WWW Document], Available at: https://cloud.google.com/dialogflow/cx/docs/concept/test-case (accessed 13.12.2021).

Amazon, n.d. Using a test framework to design better experiences with Amazon Lex [WWW Document], 2020. . Amaz. Web Serv. Available at: https://aws.amazon.com/blogs/machine-learning/using-a-test-framework-to-design-better-experiences-with-amazon-lex/ (accessed 13.12.2021).

Weizenbaum, J., 1966. ELIZA—a Computer Program for the Study of Natural Language Communication between Man and Machine. Commun ACM 9, 36–45. https://doi.org/10.1145/365153.365168

Amazon, n.d. What is Amazon Connect? - Amazon Connect [WWW Document], Available at: https://docs.aws.amazon.com/connect/latest/adminguide/what-is-amazon-connect.html (accessed 13.12.2021).

What is NGINX? How different is it from Apache (for example)? [WWW Document], n.d. . NGINX. URL https://www.nginx.com/faq/what-is-nginx-how-different-is-it-from-e-g-apache/ (accessed 12.13.21).

Woods, W.A., 1978. Semantics and Quantification in Natural Language Question Answering, in: Advances in Computers. Elsevier, pp. 1–87. https://doi.org/10.1016/S0065-2458(08)60390-3